

Task Manager API

Step 1: Create a directory for the project

C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API

Step 2: Creating a Python virtual environment

Why do we need **virtual environments**?

- To create an isolated development workspace for each project, ensuring that dependencies and package versions do not interfere with one another. Since, by default the Python interpreter installs all packages globally into a single shared folder. This can raise these potential threats;
 - **Preventing Version Conflicts:** Different projects often require different versions of the same library. For example, Project A might require Django 3.2, while Project B requires Django 5.0. Updating a global package for one project would instantly break the other.
 - **Ensuring Project Reproducibility:** You can generate a lightweight requirements.txt file listing the exact package versions used in your environment. This allows teammates or deployment servers to easily recreate your identical setup, eliminating the common "it works on my machine" problem.
 - **Protecting the Operating System:** Many operating systems (especially Linux and macOS) rely on a system-wide Python installation to run core applications. Modifying global packages can accidentally corrupt critical system functions or fail due to strict administrator permission barriers.
 - **Keeping Systems Pristine:** Keeping experimental or temporary packages confined to a single directory prevents your global workspace from becoming bloated and cluttered over time.
 - **Disposable Workspaces:** Virtual environments are entirely modular. If a package installation goes wrong or breaks your configuration, you can simply delete the environment folder and rebuild a fresh one in seconds without risking your main Python setup.

Steps:

- Open Powershell in the project directory
- Run: **python -m venv venv**
 - python Runs the Python interpreter
 - -m Runs a Python module
 - venv Python's built-in virtual environment module
 - venv Name of the folder in which the virtual environment will be created

- To activate the venv, Run: **.\venv\Scripts\Activate.ps1**
 - Possible error: "running scripts is disabled on this system." Solution;
 - Close Powershell
 - Open Powershell again as 'Administrator'
 - Run: Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
 - Run: Y
 - Run: .\venv\Scripts\Activate.ps1
 - If activation was successful then the terminal will from
PS D:\Projects\task-manager-api>
To
(venv) PS D:\Projects\task-manager-api>

Step 3: Upgrading pip

What is **pip**?

- **pip** is the standard package manager for Python, used to install, update, and manage extra libraries and dependencies that are not included in the Python standard library. It stands for "**Pip Installs Packages**" or "**Python Package Installer**" and connects directly to the [Python Package Index \(PyPI\)](#), a massive online repository of free, community-developed software.

Core Functions of pip

- **Downloads code automatically** from the internet.
- **Resolves complex dependencies** required by libraries.
- **Installs pre-built packages** like Wheels (`.whl`).
- **Integrates with virtual environments** seamlessly.

Bulk Installation with Requirements File

For large development projects, developers track dependencies inside a text file. You can download dozens of libraries simultaneously using the [pip installation documentation](#) strategy:  PIP documentation +2

```
bash
```

```
pip install -r requirements.txt
```

Use code with caution.



Essential Terminal Commands

You use `pip` through your system terminal or command prompt.  GeeksforGeeks

Task	Command	Description
Check Version	<code>pip --version</code>	Checks if <code>pip</code> is properly installed.
Install Package	<code>pip install package_name</code>	Downloads and installs the latest version.
Specific Version	<code>pip install package_name==1.5</code>	Installs a designated version layout.
Upgrade	<code>pip install --upgrade package_name</code>	Updates a library to the latest release.
Uninstall	<code>pip uninstall package_name</code>	Removes the library from your system.
List Installed	<code>pip list</code>	Shows all installed third-party libraries.

Why do we need to “upgrade pip” before installing anything, while creating a python project?

- Upgrading `pip` before installing any packages ensures **faster installation times, modern binary compatibility, and fewer dependency resolution errors**. When you install Python or create a fresh virtual environment, the system bundled version of `pip` is often months or even years outdated.

Steps:

- Pip is mostly upgraded after creating a virtual environment and then activating it, i.e.
 - # 1. Create your environment
`python -m venv .venv`
 - # 2. Activate it (Windows)
`.venv\Scripts\activate`
 - # 3. Securely upgrade pip within that environment
`python -m pip install --upgrade pip`

Note: Always use `python -m pip` rather than just running `pip install`. The `python -m` syntax guarantees you are upgrading the `pip` explicitly tied to your active Python interpreter, preventing cross-contamination of system files.

Step 4: Install FastAPI

What is FastAPI?

- **FastAPI** is a modern, open-source, high-performance web framework used for building **REST APIs with Python**. It leverages standard Python type hints to deliver exceptional speed, automated data validation, and native support for asynchronous programming.

Core Features

- **High Performance:** Delivers speeds comparable to Node.js and Go by operating on Starlette (for web routing) and Pydantic (for data validation).
- **Automatic Documentation:** Generates interactive web-browser interfaces like Swagger UI and ReDoc directly from your source code parameters.
- **Data Validation:** Catches formatting errors instantly because Pydantic parses incoming HTTP request payloads against your specified Python data types.
- **Asynchronous Execution:** Supports the standard `async/await` syntax natively via the [Asynchronous Server Gateway Interface \(ASGI\)](#) protocol.
- **Fewer Bugs:** Minimizes human-induced code developer errors by roughly 40% through extensive auto-completion and type safety safety-checks.

Steps:

- **Run:** `pip install fastapi`

Step 5: Install Uvicorn

What is Uvicorn?

- Uvicorn is a lightning-fast, asynchronous web server for Python. It acts as the bridge that listens for incoming network requests and passes them to your Python application. It is primarily used to run modern async frameworks like [FastAPI](#) and Starlette.
- Because web frameworks define how your API handles requests but lack the built-in ability to listen for HTTP network traffic, an interface like Uvicorn is essential. It conforms to the **ASGI** (Asynchronous Server Gateway Interface) standard.

Key aspects of Uvicorn include:

- **High Performance:** Uvicorn is built on top of the ultra-fast libraries `uvloop` (a drop-in replacement for the standard `asyncio` event loop) and `httptools` (for HTTP parsing).

- **Feature Support:** Unlike older, synchronous standards (like WSGI), Uvicorn handles modern web protocols like HTTP/1.1, HTTP/2, and persistent WebSockets.
- **Development Reloading:** Uvicorn includes a `--reload` flag, which is highly popular for local development as it automatically restarts the server when code changes are detected.
- **Production Deployment:** While running Uvicorn alone is great for development, it is typically used alongside process managers like [Gunicorn](#) for production deployment. Gunicorn manages the worker processes across multiple CPU cores, while Uvicorn executes the actual async application logic.

Steps:

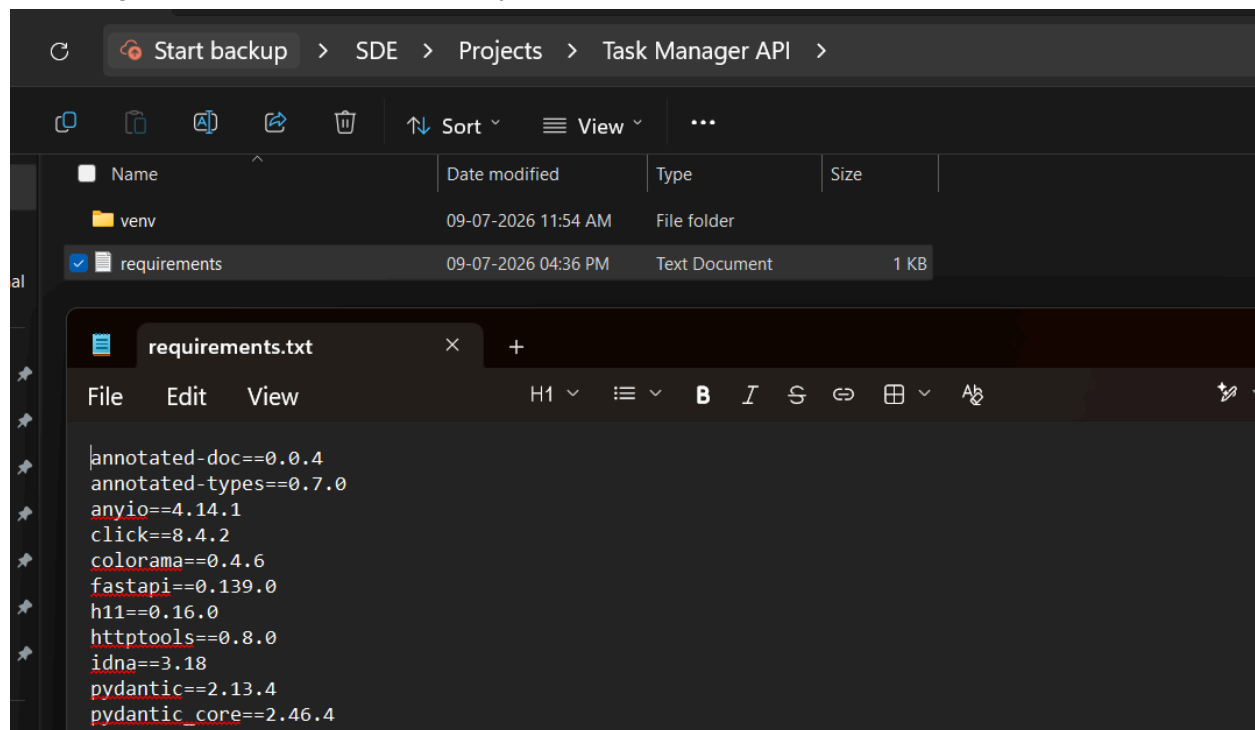
- `pip install "uvicorn[standard]"`

Step 6: Save the Dependencies

Steps:

- `pip freeze > requirements.txt`

This file gets created, which allows anyone to recreate the same environment;



Step 7: Create the Project Structure

Steps:

- Run these commands one by one:
 - **mkdir app**
 - **mkdir tests**
 - **mkdir alembic**
- Now create folders inside app:
 - **mkdir app\api**
 - **mkdir app\core**
 - **mkdir app\database**
 - **mkdir app\models**
 - **mkdir app\repositories**
 - **mkdir app\schemas**
 - **mkdir app\services**
- Now create the Python files:
 - **New-Item app\main.py -ItemType File**
 - **New-Item README.md -ItemType File**
 - **New-Item .gitignore -ItemType File**
 - **New-Item .env -ItemType File**
- If you're using Command Prompt instead of PowerShell:
 - **type nul > app\main.py**
 - **type nul > README.md**
 - **type nul > .gitignore**
 - **type nul > .env**
- Your folder should now look like:

```
task-manager-api/
|
├─ alembic/
├─ app/
|   ├── api/
|   ├── core/
|   ├── database/
|   ├── models/
|   ├── repositories/
|   ├── schemas/
|   ├── services/
|   └─ main.py
|
├─ tests/
├─ venv/
├─ .env
├─ .gitignore
├─ README.md
└─ requirements.txt
```

Step 8: Write Your First FastAPI Application

Write code inside `app/main.py`

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {
        "message": "Welcome to the Task Management API!"
    }
```

Here;

<pre>from fastapi import FastAPI</pre>	- Imports the FastAPI class.
<pre>app = FastAPI()</pre>	- Creates the FastAPI application.
<pre>@app.get("/")</pre>	- Creates an endpoint for: GET /
<pre>def read_root():</pre>	- Defines the function that runs when the endpoint is requested.
<pre> return { "message": "Welcome to the Task Management API!" }</pre>	- FastAPI automatically converts this Python dictionary into JSON.

Step 9: Run the API

In the terminal, Run: **uvicorn app.main:app --reload**

Output;

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Anurag Laumas\\Desktop\\SDE\\Projects\\Task Manager API']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [28908] using WatchFiles
INFO: Started server process [13828]
INFO: Waiting for application startup.
```

Let's understand this command:

Part	Meaning
<code>uvicorn</code>	Starts the ASGI server
<code>app.main</code>	<code>main.py</code> inside the <code>app</code> folder
<code>:app</code>	Uses the variable named <code>app</code>
<code>--reload</code>	Automatically restarts when files change

Step 10: Test the API

In browser, open the link displayed at the output; <http://127.0.0.1:8000/>

Expected output:

```
{
  "message": "Welcome to the Task Management API!"
}
```

Now to see the **Swagger UI** which is auto-generated by FastAPI;

Now open:

`http://127.0.0.1:8000/docs`



This is FastAPI's automatically generated Swagger UI.

You should see one endpoint:

`GET /`



Try clicking:

`Try it out`



Then:

`Execute`



You'll receive the JSON response directly in the browser.

Output;

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/
```

Server response

Code	Details
200	Response body <pre>{ "message": "Welcome to the Task Management API!" }</pre> Response headers <pre>content-length: 49 content-type: application/json date: Thu, 09 Jul 2026 12:42:57 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

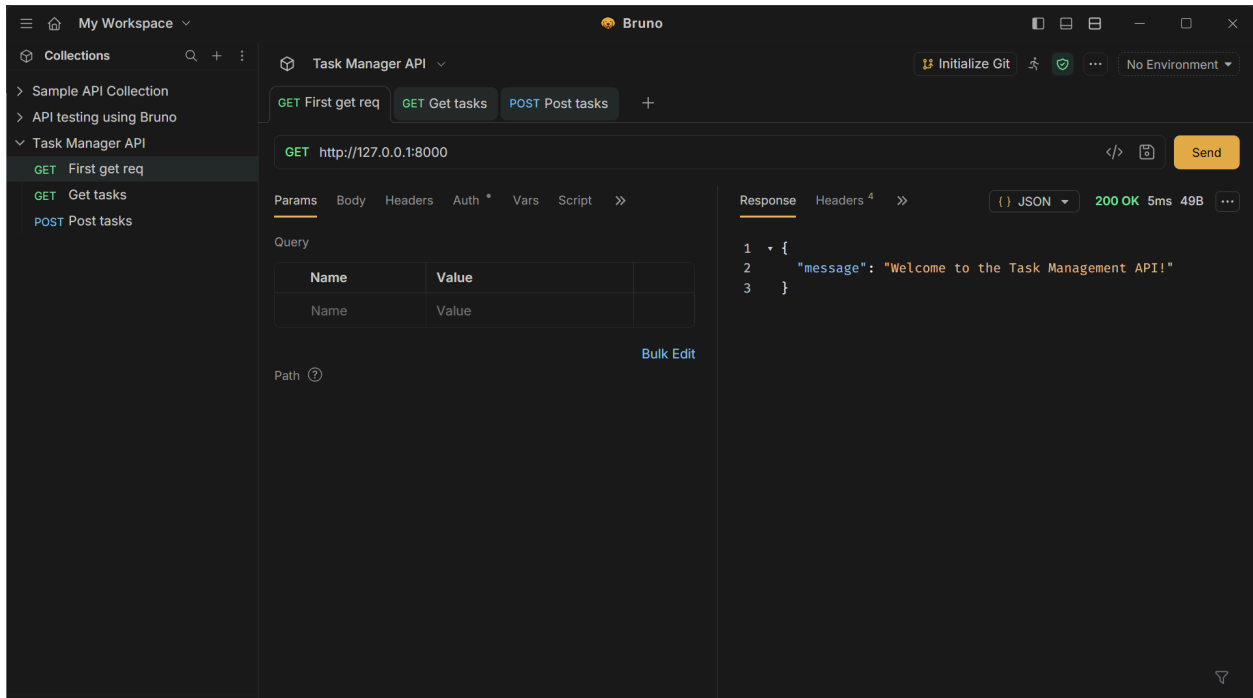
application/json

Controls Accept header

Example Value | Schema

```
"string"
```

Testing in Bruno;



After this we close the application, Run: Ctrl + C

Step 11: Creating a local repository using Git

What is Git?

- **Git** is a free, open-source **distributed version control system** designed to track changes in files and coordinate work among multiple people. Created by Linus Torvalds in 2005, it acts like a time machine for your codebase, allowing you to save snapshots of your files, view past edits, and revert to previous versions if mistakes are made.

How Git Works

Git tracks your files by utilizing three distinct local environments:

- **Working Directory:** The local folder where you are currently making active modifications to your code.
- **Staging Area:** A temporary checklist where you prepare and approve specific changes for the next save point.
- **Local Repository:** The final database on your machine containing the complete history of your saved files.

Key Core Concepts

- **Commit:** A saved snapshot of your staged changes with a unique ID.
- **Branch:** An isolated workspace allowing you to develop new features safely without affecting the main code.
- **Merge:** The process of combining changes from separate branches back together.
- **Repository (Repo):** The project folder containing all your files and their entire revision history.

Common Git Commands

You interact with Git locally using standard terminal commands:

- `git init`: Turns an existing folder into a brand new Git repository.
- `git status`: Checks which files have been modified or staged.
- `git add`: Moves your local file changes into the staging area.
- `git commit -m "message"`: Saves your staged snapshot securely to your project history.

Git vs. GitHub

It is common to confuse Git with GitHub, but they are entirely separate tools:

- **Git:** The local command-line application that tracks your file history offline.
- **GitHub:** A cloud-based platform that hosts Git repositories online, allowing teams to share and collaborate on code. Alternate hosting platforms include [GitLab](#) and [Bitbucket](#).

Steps:

- **Initialize Git**, Run: **git init**

Output: Initialized empty Git repository in

C:/Users/Anurag Laumas/Desktop/SDE/Projects/Task Manager API/.git/

- Create the **.gitignore**

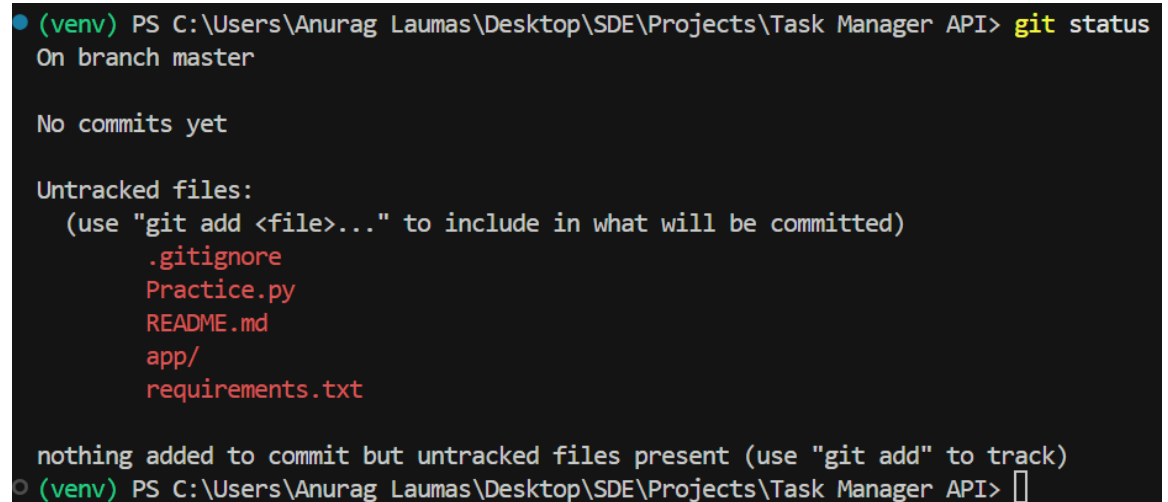
Open the .gitignore file in the root folder of the project and add these;

```
venv/  
__pycache__/  
*.pyc  
.env  
.vscode/
```

This tells Git not to track these files and folders.

- **Check Git Status**, Run: **git status**

Output:

A terminal window with a dark background and light-colored text. The prompt is '(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API>'. The command 'git status' has been executed. The output shows 'On branch master', 'No commits yet', and a list of 'Untracked files: .gitignore, Practice.py, README.md, app/, requirements.txt'. It also states 'nothing added to commit but untracked files present (use "git add" to track)'.

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git status  
On branch master  
  
No commits yet  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    .gitignore  
    Practice.py  
    README.md  
    app/  
    requirements.txt  
  
nothing added to commit but untracked files present (use "git add" to track)  
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> █
```

This is a list of untracked files.

We notice that `venv/` is **not** listed because `.gitignore` is working.

- **Configure Git (One-Time Setup):**

If it is our first time using Git, then we need to set our identity, Run:

```
git config --global user.name "Paras"
```

```
git config --global user.email "paraslaumas17@gmail.com"
```

- **To verify, Run: `git config --list`**

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git config --list
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Paras
:
```

- **Stage Files, Run: `git add .`**

This stages all project files (except those ignored by .gitignore).

To verify, Run: **`git status`**

Output:

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   Practice.py
        new file:   README.md
        new file:   app/main.py
        new file:   requirements.txt

(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API>
```

- **Create Your First Commit, Run: `git commit -m "Initial FastAPI project setup"`**

Output:

```
new file:   requirements.txt
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git commit -m "Initial FastAPI project setup"
[master (root-commit) af81ac7] Initial FastAPI project setupanager API
5 files changed, 14 insertions(+)
create mode 100644 .gitignore
create mode 100644 Practice.py
create mode 100644 README.md
create mode 100644 app/main.py
create mode 100644 requirements.txt
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API>
```

To verify, Run: `git log --oneline`

Output:

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git log --oneline
af81ac7 (HEAD -> master) Initial FastAPI project setup
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API>
```

11/07/26

Step 12: Creating a GitHub repository using Git

26/02/24

[B] For Remote repositories (i.e. on someone else's server/ computer)

[Use GitHub]

`git remote add --name --url` → After creating a new repo. in our GitHub acc;

(origin)

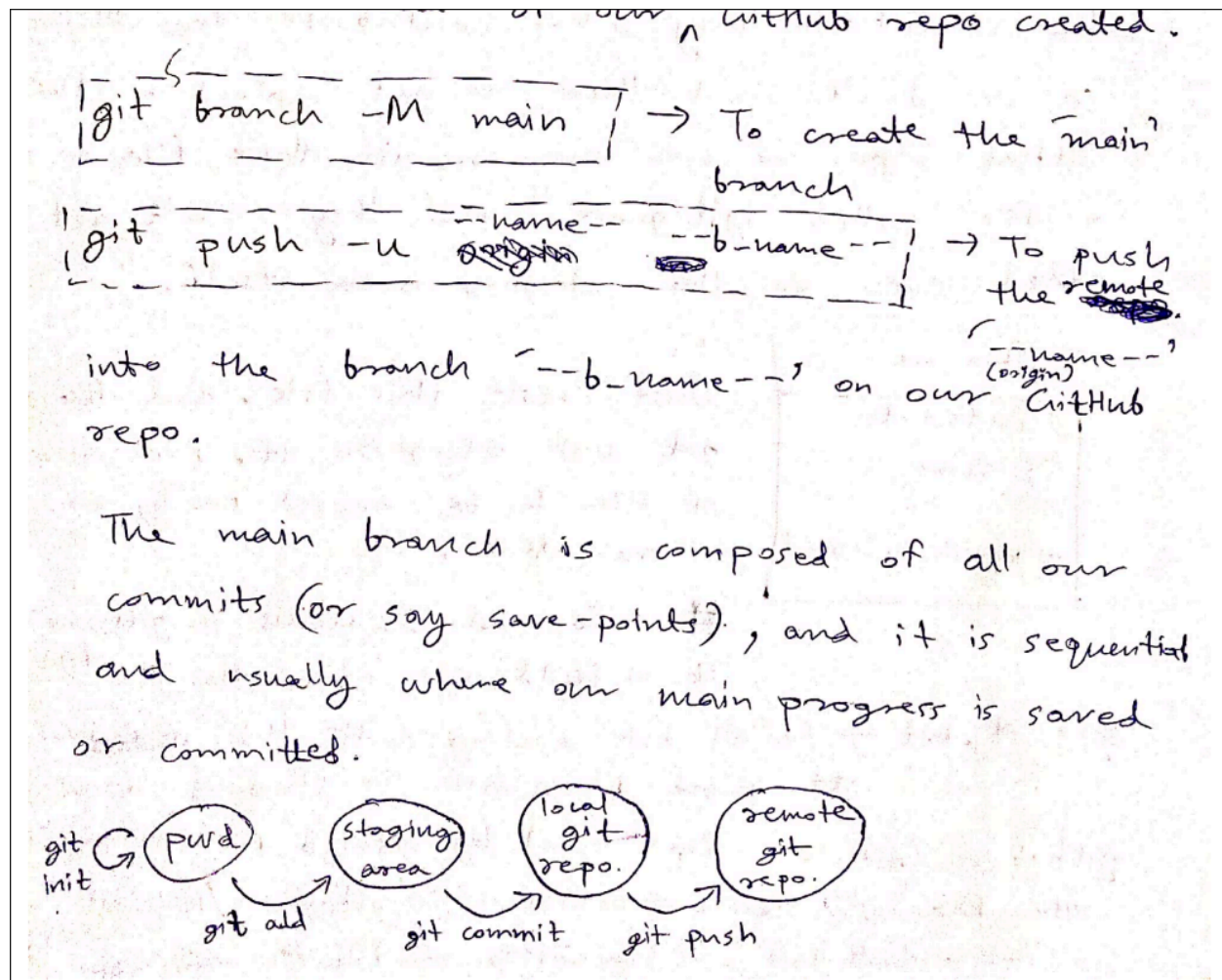
we use this command to create a 'remote'.

The '-name-' is generally kept as 'origin' to increase the readability to other programmers.

'url' is the add^r of our ^{new} GitHub repo created.

`git branch -M main` → To create the 'main' branch

`git push -u --name --b-name` → To push the ~~remote~~ remote



Hence, Run:

```
git remote add paras_1 https://github.com/paraslaumas111/Task-Manager-API.git
```

```
git branch -M main
```

```
git push -u paras_1 main
```

-----Commit 1 Complete-----

Step 13: Creating a Python Package

What is a Python Package?

- A **Python package** is a directory (folder) that structures and organizes related [Python modules](#) (individual `.py` files) into a single, manageable hierarchy. By bundling your code into a package, you can easily scale a codebase, avoid module name conflicts, and share reusable tools with other developers.

Package Structure Explained

At its core, a package is just a folder on your computer. However, to recognize it natively as an importable package, Python traditionally looks for a special file named `__init__.py` inside that folder.

A standard package structure looks like this:

text

```
my_game_package/           # Top-level package folder
|
├── __init__.py             # Initializes the package (can be empty)
├── characters.py           # A module containing character classes
└── levels/                 # A sub-package for deeper organization
    ├── __init__.py         # Initializes the sub-package
    └── boss_fight.py        # A module inside the sub-package
```

Modules: Individual `.py` files containing functions, classes, and variables.

Sub-packages: Nested directories inside the main package folder to group code even further.

`__init__.py`: This file executes automatically when the package is imported. It can contain initialization logic or be left completely blank. *(Note: Since Python 3.3, you can create "implicit namespace packages" without this file, but keeping it remains the standard practice for regular packages).*

How to Use a Package

Once a package is created or downloaded, you bring its functionality into your script using the `import` keyword combined with dot (`.`) notation.  Programiz +1

python

```
# Option 1: Import a specific module from the package
import my_game_package.characters

# Option 2: Import from a deeply nested sub-package
from my_game_package.levels import boss_fight

# Option 3: Import a specific tool or function directly
from my_game_package.characters import PlayerHero
```

Local Packages vs. Published Packages

1. **Local Packages:** Directories you create yourself on your machine to structure a complex software project you are actively building.
2. **Published Packages:** Code bundles uploaded by other developers to a public repository like the [Python Package Index \(PyPI\)](#). You can download and install these third-party toolkits using package managers like `pip` or `uv`.
 - *Examples of popular published packages:* [Pandas](#) (data analysis), [Django](#) (web development), and [Matplotlib](#) (data visualization).

Summary Checklist: Modules vs. Packages vs. Libraries

To keep the terminology straight, use this general rule of thumb:

-  **Module:** A single Python file (`.py`).
-  **Package:** A folder containing modules and an `__init__.py` file.
-  **Library:** An umbrella term for a large collection of related packages and modules bundled together (like the [Python Standard Library](#)

Steps:

- In Python, a directory becomes a package when it contains an `__init__.py` file.

Run: **New-Item app\api__init__.py -ItemType File**

(Or create it manually in VS Code)

Output -

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> New-Item app\api\__init__.py -ItemType File

Directory: C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API\app\api

Mode                LastWriteTime         Length Name
----                -
-a-----         12-07-2026  10:51 AM              0 __init__.py
```

Step 14: Creating a Router

What is a Router in a Python Project?

- In a Python web project (such as FastAPI or Django), a **router** is a component that directs incoming web requests to the specific function or code block meant to handle them. It acts as a traffic controller, matching URL paths and HTTP methods to your predefined code.

Key Functions of a Router

- **Traffic Direction:** When a user accesses a URL (e.g., `GET /users/42`), the router intercepts the request and maps it to the specific view or function assigned to that route.
- **Code Organization:** Instead of putting all your endpoints in one massive file, routers let you break your project into smaller, logical modules (e.g., a `users` router and an `orders` router).
- **Prefixing & Middleware:** Routers allow you to apply global prefixes (like `/api/v1`) or specific security/authentication rules to an entire group of endpoints at once.

Examples in Popular Frameworks

- **FastAPI:** Uses the `APIRouter` class to group path operations and structure apps across multiple files. You can read more about it in the [FastAPI APIRouter Reference](#).

- **Django:** Uses a URL configuration (often defined in a `urls.py` file) to link URLs to view functions, and the Django Rest Framework uses `Routers` to automatically generate API URLs. You can explore how this works in the [Django REST Framework Routers Guide](#).
- **Flask:** Uses the `@app.route` decorator directly above Python functions to achieve the same routing behavior.

Steps:

- Run: **New-Item app\api\tasks.py -ItemType File**

Output -

```
PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> New-Item app\api\tasks.py -ItemType File
Directory: C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API\app\api

Mode                LastWriteTime         Length Name
----                -
-a----           12-07-2026  11:03 AM              0 tasks.py

PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API\venv\Scripts\Activate.ps1")
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API>
```

- Code for the Router;

```
from fastapi import APIRouter

router = APIRouter(
    prefix="/tasks",
    tags=["Tasks"]
)

@router.get("/")
def get_all_tasks():
    return {
        "message": "Get all tasks"
    }
```

```
@router.post("/")
def create_task():
    return {
        "message": "Create a task"
    }
```

Here;

<pre>from fastapi import APIRouter</pre>	- Imports FastAPI's router class. - Instead of attaching endpoints directly to the application, we attach them to a router.
<pre>router = APIRouter(</pre>	- Creates a router.
<pre> prefix="/tasks",</pre>	- Every endpoint in this router automatically starts with: <pre>/tasks</pre> So: <pre></> Python</pre> <pre>@router.get("/")</pre> actually becomes: <pre>GET /tasks</pre>
<pre> tags=["Tasks"]</pre>	- Groups endpoints together in Swagger UI. - Without tags, everything appears in one large list.
<pre> return { "message": "Welcome to the Task Management API!" }</pre>	- FastAPI automatically converts this Python dictionary into JSON.

- Updating the main.py;

```
from fastapi import FastAPI

from app.api.tasks import router as task_router

app = FastAPI(
    title="Task Management API",
    version="1.0.0"
)

@app.get("/")
def root():
    return {
        "message": "Welcome to the Task Management API!"
    }

app.include_router(task_router)
```

Here;

<pre>from app.api.tasks import router as task_router</pre>	- Imports the router from tasks.py and gives it a descriptive alias.
<pre>app.include_router(task_router)</pre>	- Registers all routes defined in that router with the main application.

Step 15: Running & Testing

To Run: `uvicorn app.main:app --reload`

Output on URL: <http://127.0.0.1:8000/>

```
{"message": "Welcome to the Task Management API!"}
```

To test Output on Swagger UI, URL: <http://127.0.0.1:8000/docs>

Task Management API 1.0.0 OAS 3.1

/openapi.json

default

GET

/ Root

Tasks

GET

/tasks/ Get All Tasks

POST

/tasks/ Create Task

Upon choosing "Try it out" option for each route;

GET: http://127.0.0.1:8000/

200

Response body

```
{
  "message": "Welcome to the Task Management API!"
}
```

Response headers

GET: http://127.0.0.1:8000/tasks

200

Response body

```
{
  "message": "Get all tasks"
}
```

Response headers

POST: http://127.0.0.1:8000/tasks

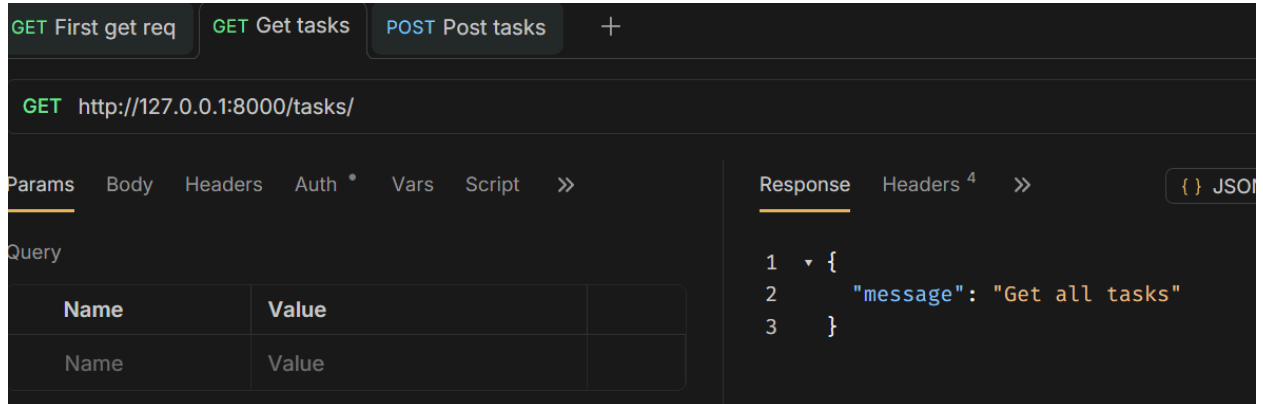
200

Response body

```
{
  "message": "Create a task"
}
```

Response headers

Testing output on Bruno;



Experimentation;

a) Adding this code in main.py; (Add another http request "post" to root endpoint)

```
@app.post("/")

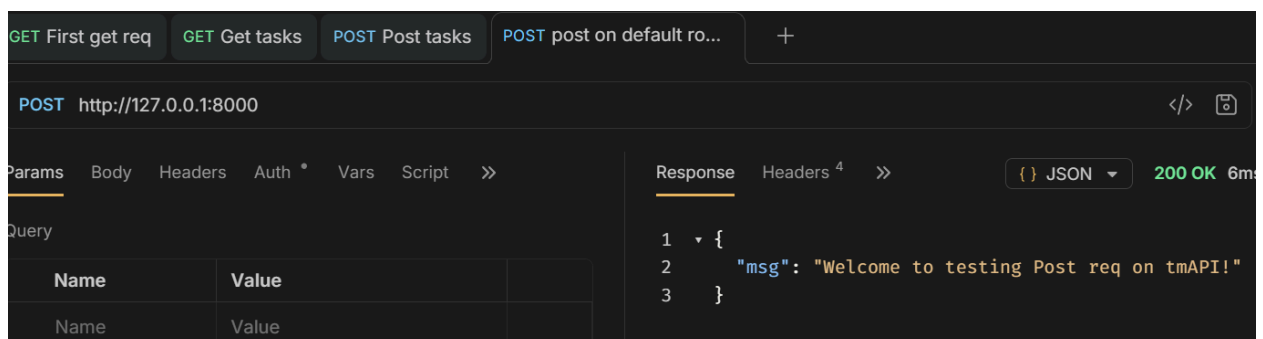
def root():

    return {

        "msg": "Welcome to testing Post req on tmAPI!"

    }
```

- Testing output on Bruno;



b) Adding this code in tasks.py (Add another endpoint: /tasks/papa, /tasks/mama)

```
@router.get("/papa")

def get_all_tasks():

    return {

        "Yo": "Imma Hustler"

    }

@router.post("/mama")

def create_task():

    return {

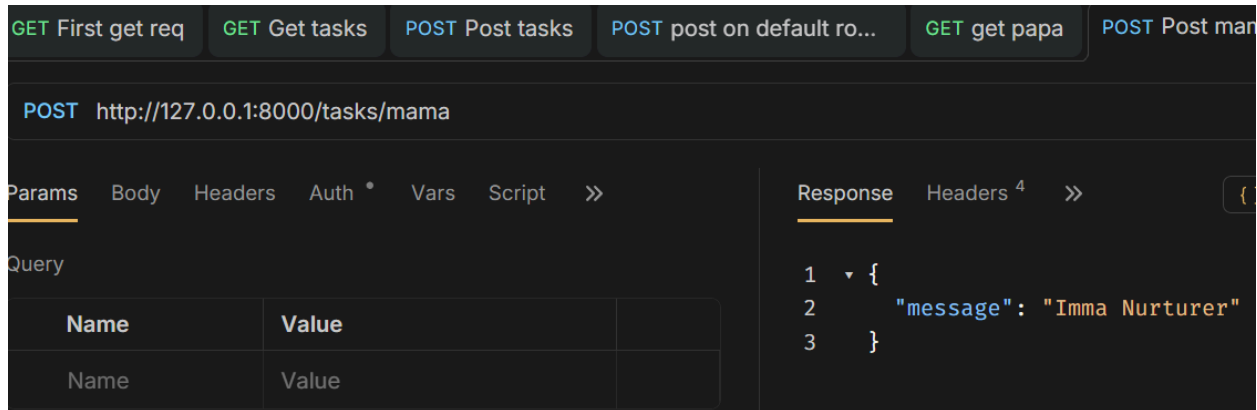
        "message": "Imma Nurturer"

    }
```

- Testing output on Bruno;

The screenshot shows the Bruno API client interface. At the top, there are tabs for different requests: "GET First get req", "GET Get tasks", "POST Post tasks", "POST post on default ro...", "GET get papa", and "POST Pos". The "GET get papa" tab is selected. Below the tabs, the request details are shown: "GET http://127.0.0.1:8000/tasks/papa". The interface is split into two main sections: "Params" on the left and "Response" on the right. The "Params" section has tabs for "Query", "Body", "Headers", "Auth", "Vars", and "Script". The "Response" section has tabs for "Response", "Headers", and "Script". The "Response" tab is selected, showing the response body:

```
{
  "Yo": "Imma Hustler"
}
```



Step 16: Pushing the code to GitHub Repository

Since we had already;

- Initialized the git
- Created the remote
- Created our identity
- Linked the Github branch

So we need to repeat these steps, and all we needed to do was;

- **git add .**
- **git commit -m "Describe what changes you made"**
- **git push remote_name main**

Since I committed some mistakes, lets see the output;

No need to initialize git again;

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git init
Reinitialized existing Git repository in C:/Users/Anurag Laumas/Desktop/SDE/Projects/Task Manager API/.git/
```

To view all updates which are left to be committed: **Git status**

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git status
On branch main
Your branch is up to date with 'paras_1/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Practice.py
        modified:   app/main.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        app/api/

no changes added to commit (use "git add" and/or "git commit -a")
```

I adding each file to the staging area individually, eg. **git add .\Practice.py .\app\main.py**
Because of which I was left with some files everytime

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git add .\Practice.py .\app\main.py
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git status
On branch main
Your branch is up to date with 'paras_1/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   Practice.py
        modified:   app/main.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        app/api/
```

Instead I should have done: **git add .**

Next step is to commit the changes: **git commit -m "Created package and Router"**

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git commit -m "Created package and Router"
[main 5b7c554] Created package and Router
4 files changed, 116 insertions(+), 3 deletions(-)
create mode 100644 app/api/__init__.py
create mode 100644 app/api/tasks.py
```

Verifying using: **git log --oneline**

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git log --oneline
5b7c554 (HEAD -> main) Created package and Router
af81ac7 (paras_1/main) Initial FastAPI project setup
```

Then to push the updates to the repo, I used this: **git push -u paras_1 main**

Which resulted in error;

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git push -u paras_1 main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.21 KiB | 414.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote: fatal error in commit_refs
To https://github.com/paraslaumas111/Task-Manager-API.git
 ! [remote rejected] main -> main (failure)
error: failed to push some refs to 'https://github.com/paraslaumas111/Task-Manager-API.git'
```

Instead I should use: **git push paras_1 main**

```
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API> git push paras_1 main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.21 KiB | 414.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/paraslaumas111/Task-Manager-API.git
   af81ac7..5b7c554  main -> main
(venv) PS C:\Users\Anurag Laumas\Desktop\SDE\Projects\Task Manager API>
```

Theory Questions -

What is the difference between FastAPI() and APIRouter()?

FastAPI() and APIRouter() serve different purposes in structuring a web application. FastAPI() is the main application instance, while APIRouter() is a mini-application used to organize routes. 

Key Differences

- **Scope:** FastAPI() manages the entire application. APIRouter() manages a specific subset of routes.
- **Initialization:** You create only one FastAPI() instance. You can create multiple APIRouter() instances.
- **Server Execution:** Uvicorn runs the FastAPI() instance. Uvicorn cannot run an APIRouter() directly.
- **Global Configuration:** FastAPI() handles global settings like title, version, and lifespan events. APIRouter() handles path prefixes and tags for grouping. 

Why do we use routers instead of putting all endpoints in main.py?

- Putting all endpoints in `main.py` works for tiny apps, but becomes unmanageable as your project grows. Using routers (like APIRouter in FastAPI or Blueprints in Flask) solves this by organizing your code efficiently.

Key Benefits of Using Routers

- **Maintainability:** Code stays clean, organized, and easy to read.
- **Separation of Concerns:** Group related routes (e.g., `/users`, `/products`) into dedicated files.
- **Collaboration:** Multiple developers can work on different feature files simultaneously without git conflicts in `main.py`.
- **Reusability:** Apply prefixes (like `/api/v1`), tags, and dependencies to an entire group of routes at once.
- **Scalability:** Easily add hundreds of endpoints without creating a massive, thousands-of-lines-long single file.

Structural Comparison

Bad: Everything in `main.py`

text

```
project/
└─ main.py ✗ (Contains 500+ lines of mixed user, product, and auth logic)
```

Use code with caution.

Good: Using Routers

text

```
project/
├─ app/
│   └─ routers/
│       ├── users.py
│       ├── products.py
│       └─ auth.py
└─ main.py ✓ (Clean; only imports and registers the routers)
```

Why is `POST /tasks` considered more RESTful than `POST /createTask`?

`POST /tasks` is considered more RESTful because REST (Representational State Transfer) emphasizes using URIs to represent **nouns** (resources or collections of resources) and HTTP methods to define the **verbs**.

- `POST /tasks`: Targets the `tasks` collection directly. The HTTP verb tells the server *what* to do, and the noun tells it *where* to do it. The server creates a new task within that collection.
- `POST /createTask`: Represents Remote Procedure Call (RPC) thinking. It bakes the action directly into the URI, which violates REST constraints where actions belong strictly to HTTP methods.



What is an API?

Imagine you're in a restaurant.

- You are the client.
- The waiter is the API.
- The kitchen is the server.

The waiter takes your order, gives it to the kitchen, brings the food back, and never exposes the kitchen directly.

Similarly:

Browser / Mobile App

|



FastAPI

|



PostgreSQL

The client never talks directly to the database—it talks to the API.



What is REST?

REST stands for Representational State Transfer.

You don't need to memorize the name. Instead, remember the idea:

A REST API organizes resources (like tasks, users, or products) and uses standard HTTP methods to interact with them.

Our resource is:

Task

We will build endpoints like:

GET /tasks

POST /tasks

GET /tasks/{task_id}

PUT /tasks/{task_id}

DELETE /tasks/{task_id}

Notice the URL is based on the resource, not the action.

Good:

POST /tasks

Bad:

POST /createTask



HTTP Methods

Method	Purpose	Example
GET	Read data	Get all tasks
POST	Create data	Create a task
PUT	Replace an entire resource	Replace a task
PATCH	Update part of a resource	Change only the priority
DELETE	Remove data	Delete a task

A good REST API uses these methods consistently.



Status Codes

These are numbers returned by the server to indicate the outcome.

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized

403 Forbidden

404 Not Found

500 Internal Server
Error

We'll start using these as we build endpoints.

Why Use Routers?

Currently, everything is in `main.py`.

That works for tiny projects but becomes unmanageable as the project grows.

Instead of:

`main.py`

containing hundreds of routes, we'll organize them.

Our structure becomes:

`app/`

|

|— `api/`

| |— `tasks.py`

| |— `__init__.py`

|

|— `main.py`

-----Commit 2 Complete-----

Step 17: Validating data in Request Body using Pydantic

What is a Request Body?

So far, our `POST /tasks` endpoint didn't accept any data.

Imagine a user wants to create a task:

```
Title: Complete FastAPI Lesson
Description: Finish Lesson 3
Priority: High
```

Where does this data go?

- It goes in the **request body**.

When a client (browser, mobile app, Bruno, etc.) sends a request to the server, it can include:

- URL
- HTTP Method
- Headers
- **Body**

For a POST request, the body usually contains JSON.

What is JSON?

JSON stands for **JavaScript Object Notation**.

It is simply a text format for exchanging data. Example:

```
</> JSON

{
  "title": "Complete FastAPI Lesson",
  "description": "Finish Lesson 3",
  "priority": "High"
}
```

Notice:

- Curly braces `{}` represent an object.
- **Keys are always strings.**
- Strings use double quotes `"`.
- Key-value pairs are separated by commas.

Why Validate Data?

Imagine someone sends:

```
</> JSON

{
  "title": "",
  "description": "abc",
  "priority": "SuperHigh"
}
```

Should we accept this?

No.

A good API protects itself from invalid data.

Validation ensures:

- Required fields are present.
- Data types are correct.
- Values follow expected rules.

Without validation, your database would quickly fill with bad data.

What is Pydantic?

Pydantic is a Python library that validates data using Python type hints.

FastAPI uses Pydantic internally.

Instead of manually checking every field like this in the code:

```
</> Python

if title == "":
    ...
```

You define a schema, and FastAPI validates it automatically.

Steps:

- Create a new file, Run: **New-Item app\schemas\task.py -ItemType File**
- In this file app/schemas/task.py, write this code:

```
from pydantic import BaseModel

class TaskCreate(BaseModel):
    title: str
    description: str
    priority: str
```

Here,

<pre>from pydantic import BaseModel</pre>	- Imports Pydantic's base class. - Every schema inherits from BaseModel.
<pre>class TaskCreate(BaseModel):</pre>	- Defines a schema named TaskCreate. - By convention: TaskCreate → Used when creating a task. - Later we'll have TaskUpdate, TaskResponse, etc.
<pre> title: str</pre>	- Means: - The field is required. - It must be a string. - Similar for; <pre> description: str priority: str</pre>

- Updating our Router file: app/api/tasks.py

```
from fastapi import APIRouter, status

from app.schemas.task import TaskCreate

router = APIRouter(
    prefix="/tasks",
    tags=["Tasks"]
)

@router.get("/")
def get_all_tasks():
    return {
        "Msg": "Get All Tasks"
    }

@router.post("/", status_code=status.HTTP_201_CREATED)
def create_task(task: TaskCreate):
    return {
        "message": "A new task created successfully",
        "task": task
    }

@router.get("/count")
def get_task_count():
    return {
        "count": 0
    }

@router.get("/health")
def health():
    return {
        "status": "Healthy"
    }
```

Here,

Importing `status`

```
</> Python
from fastapi import status
```

Instead of writing:

```
</> Python
status_code=201
```

we write:

```
</> Python
status.HTTP_201_CREATED
```

This is more readable and avoids "magic numbers."

The Function Parameter

```
</> Python
task: TaskCreate
```

Run

This is the magic of FastAPI.

It means:

"Expect the request body to match the `TaskCreate` schema."

FastAPI will:

1. Read the JSON.
2. Validate it.
3. Convert it into a Python object.
4. Pass it to the function.

Returning the Object

`</>` Python

```
return {  
    "message": "Task created successfully!",  
    "task": task  
}
```

FastAPI knows how to convert a Pydantic model back into JSON.

Now to run: **uvicorn app.main:app --reload**

Output -

Task Management API 1.0.0 OAS 3.1

[/openapi.json](#)

default ^

GET / Root

POST / Root

Tasks ^

GET /tasks/ Get All Tasks

POST /tasks/ Create Task

GET /tasks/count Get Task Count

GET /tasks/health Health

Schemas

Inside Schemas -

Schemas

HTTPValidationError ^ Collapse all object

- detail ^ Collapse all array<object>
 - Items ^ Collapse all object
 - loc* ^ Collapse all array<(string | integer)>
 - Items ^ Collapse all (string | integer)
 - Any of ^ Collapse all (string | integer)
 - #0 string
 - #1 integer
- msg* string
- type* string
- input any
- ctx object

TaskCreate ^ Collapse all object

- title* string
- description* string
- priority* string

ValidationError ^ Collapse all object

- loc* ^ Collapse all array<(string | integer)>
 - Items ^ Collapse all (string | integer)
 - Any of ^ Collapse all (string | integer)
 - #0 string
 - #1 integer
- msg* string
- type* string
- input any
- ctx object

First we need to input values;

Tasks

GET

/tasks/ Get All Tasks

^

POST

/tasks/ Create Task

^

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "title": "asdasd",
  "description": "qweqw",
  "priority": "sadqweq"
}
```

Execute

Clear

Response;

Responses

Curly

```
curl -X 'POST' \
'http://127.0.0.1:8000/tasks/' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "title": "asdasd",
  "description": "qweqw",
  "priority": "sadqweq"
}'
```

Request URL

```
http://127.0.0.1:8000/tasks/
```

Server response

Code	Details
201	Response body <pre>{ "message": "A new task created successfully", "task": { "title": "asdasd", "description": "qweqw", "priority": "sadqweq" } }</pre> Response headers <pre>content-length: 114 content-type: application/json date: Mon, 13 Jul 2026 12:51:22 GMT server: unicorn</pre>

Responses

Code	Description	Links
201	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

If I input input values which violates the schema design that we have set using pydantic;

GET /tasks/ Get All Tasks

POST /tasks/ Create Task

Parameters

No parameters

Request body required

application/json

Edit Value Schema

```
{  "title": "asdasd",}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  http://127.0.0.1:8000/tasks/' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "title": "asdasd",}
```

In that case we receive validation error message in response;

Responses

Curl

```
curl -X 'POST' \  http://127.0.0.1:8000/tasks/' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "title": "asdasd",}
```

Request URL

http://127.0.0.1:8000/tasks/

Server response

Code

Details

422

Error: Unprocessable Content

Response body

```
{  "detail": [    {      "type": "json_invalid",      "loc": [        "body",        21      ],      "msg": "JSON decode error",      "input": {},      "ctx": {        "error": "Illegal trailing comma before end of object"      }    }  ]}
```

Response headers

```
content-length: 153  content-type: application/json  date: Mon, 13 Jul 2026 12:53:28 GMT  server: uvicorn
```

Responses

Code	Description	Links
201	Successful Response	No links

The other get request is working like a regular request;

The screenshot shows a REST client interface with the following sections:

- Parameters:** A tab labeled "Parameters" with a "Cancel" button. It displays "No parameters". Below it are "Execute" and "Clear" buttons.
- Responses:** A section containing:
 - Curl:** A code block with the command:

```
curl -X 'GET' \
  'http://127.0.0.1:8000/tasks/count' \
  -H 'accept: application/json'
```
 - Request URL:** A text field containing `http://127.0.0.1:8000/tasks/count`.
 - Server response:** A table with two columns: "Code" and "Details".

Code	Details
200	Response body<pre>{ "count": 0 }</pre> Download Response headers<pre>content-length: 11 content-type: application/json date: Mon, 13 Jul 2026 12:55:38 GMT server: uvicorn</pre>
- Responses:** A table with three columns: "Code", "Description", and "Links".

Code	Description	Links
200	Successful Response	No links

Even if I don't omit any value, but enter the value of unmatched data-type as mentioned in the schema, then also error code occurs;

The screenshot shows a schema editor with two tabs: "Edit Value" and "Schema". The "Schema" tab is active, displaying a JSON schema:

```
{
  "title": "string",
  "description": 123,
  "priority": True
}
```

Server response	
Code	Details
422	Error: Unprocessable Content
	Response body
	<pre>{ "detail": [{ "type": "json_invalid", "loc": ["body", 59], "msg": "JSON decode error", "input": {}, "ctx": { "error": "Expecting value" } }] }</pre>
	Response headers

Using Bruno;

POST Untitle •

POST Post using req body

GET Untitle •

+

POST http://127.0.0.1:8000/tasks/ </> 📄 Send

Params

Body *

Headers >>

JSON ▾

Prettify

Response >>

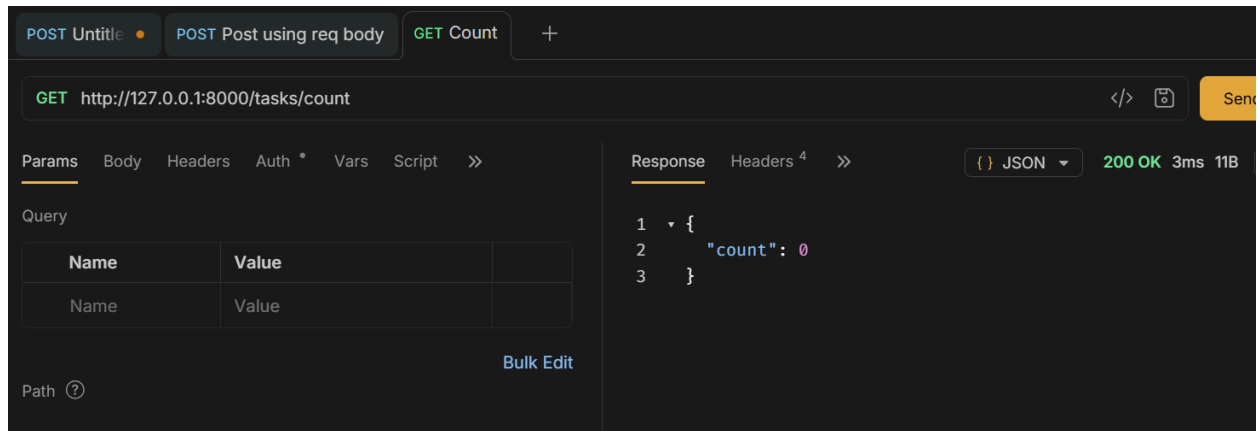
{ } JSON ▾

201 Created 3ms 110B

...

```
1 {
2   "title": "string",
3   "description": "sada",
4   "priority": "sada"
5 }
```

```
1 {
2   "message": "A new task created successfully",
3   "task": {
4     "title": "string",
5     "description": "sada",
6     "priority": "sada"
7   }
8 }
```



Now, Pushing the updates on to our GitHub repo.;

- **git add .**
- **git commit -m "Added data Validation in Request body using Pydantic"**
- **git push paras_1 main**

Then verifying using: **git log**

```
commit 951e5adfdfac353c55545aafcc2aa335213bc02e (HEAD -> main, paras_1/main)
Author: Paras <paraslaumas17@gmail.com>
Date:   Mon Jul 13 18:49:44 2026 +0530

    Added data Validation in Request body using Pydantic

commit 5b7c554a7e57d268e2e65178a4721355f8b92457
Author: Paras <paraslaumas17@gmail.com>
Date:   Sun Jul 12 12:23:34 2026 +0530

    Created package and Router

commit af81ac768567fdd0fd810a39d90595f95eb85663
Author: Paras <paraslaumas17@gmail.com>
Date:   Fri Jul 10 14:48:17 2026 +0530

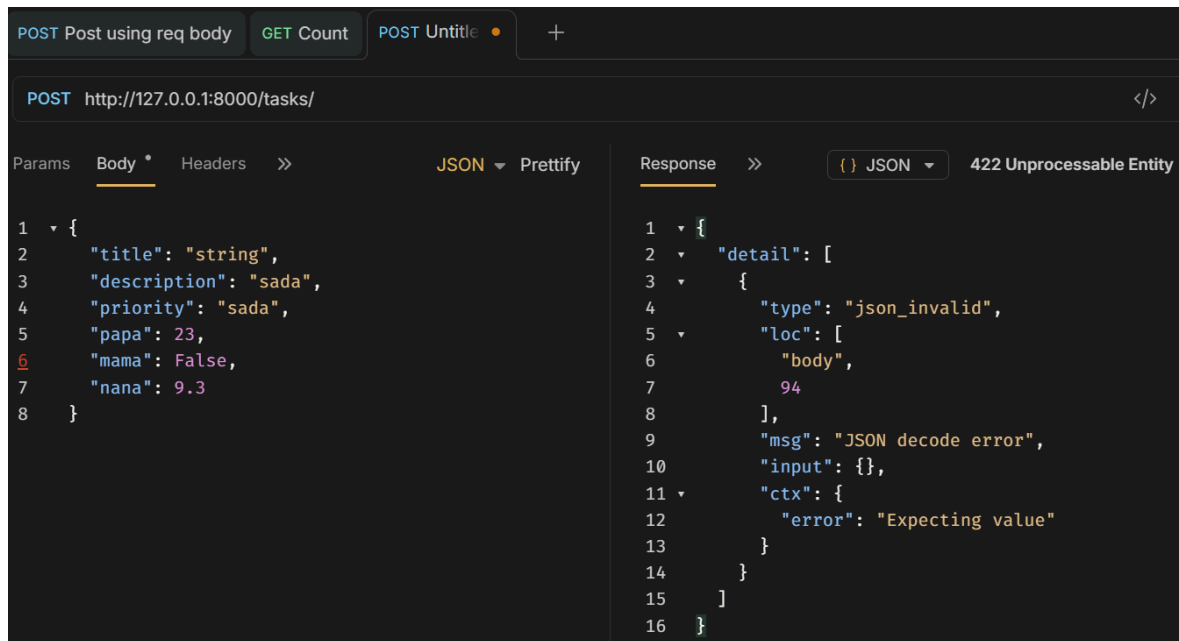
    Initial FastAPI project setup
~
~
~
~
```

For experimentation I tried to add these elements in the body validation code for pydantic;

```
from pydantic import BaseModel
```

```
class TaskCreate(BaseModel):
    title: str
    description: str
    priority: str
    papa: int
    mama: bool
    nana: float
```

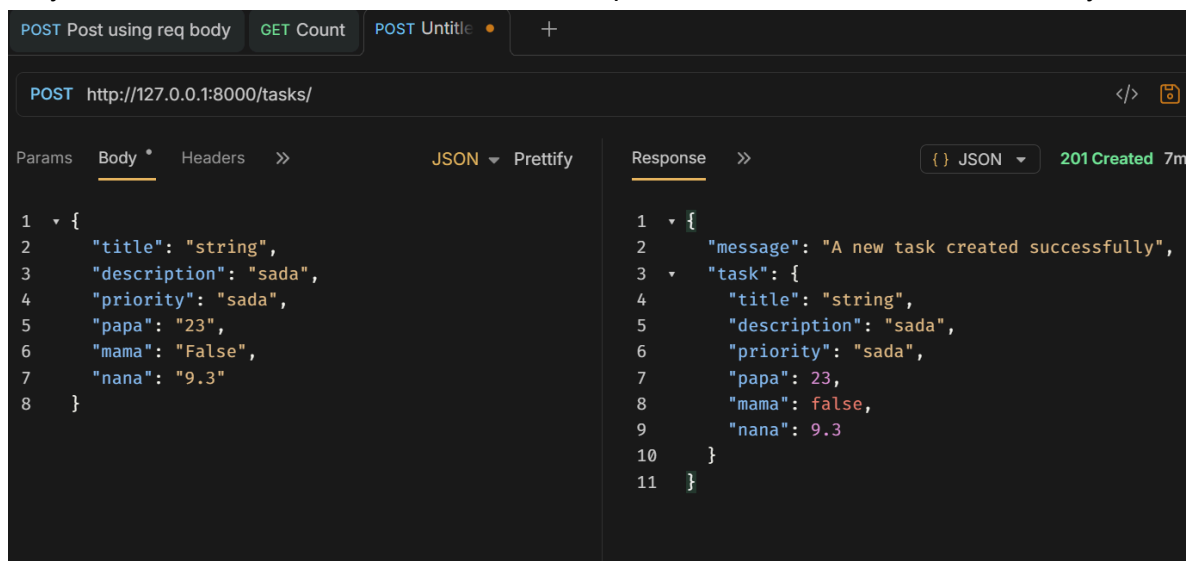
While testing output, I put the int, float and bool values without quotations, then error occurred



The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8000/tasks/`. The request body is a JSON object with the following fields: `title` (string), `description` (string), `priority` (string), `papa` (integer 23), `mama` (boolean False), and `nana` (float 9.3). The response is a 422 Unprocessable Entity error with the following JSON body:

```
{
  "detail": [
    {
      "type": "json_invalid",
      "loc": [
        "body",
        94
      ],
      "msg": "JSON decode error",
      "input": {},
      "ctx": {
        "error": "Expecting value"
      }
    }
  ]
}
```

Only when I even mentioned them also inside quotations, then code ran successfully;



The screenshot shows the same REST client interface with the same POST request to `http://127.0.0.1:8000/tasks/`. In this case, the request body values are quoted: `"papa": "23"`, `"mama": "False"`, and `"nana": "9.3"`. The response is a 201 Created status with the following JSON body:

```
{
  "message": "A new task created successfully",
  "task": {
    "title": "string",
    "description": "sada",
    "priority": "sada",
    "papa": 23,
    "mama": false,
    "nana": 9.3
  }
}
```

Theory questions

What is the difference between query parameters and a request body?

- The core difference is that **query parameters are appended directly to the URL string** to filter or sort data, whereas a **request body carries a structured data payload hidden inside the HTTP request stream** to create or modify data.

Quick Comparison

Feature	Query Parameters	Request Body
Location	Inside the URL (after the <code>?</code>)	Inside the HTTP request payload
Visibility	Fully visible in the browser address bar and logs	Hidden from the URL
Primary Use	Filtering, sorting, and pagination	Creating or updating complex resources
Data Structure	Flat key-value pairs (<code>key=value</code>)	Hierarchical, nested data (JSON, XML)
Data Size	Strictly limited by maximum URL length constraints	Virtually unlimited data payload capacity
HTTP Methods	Primarily <code>GET</code>	Typically <code>POST</code> , <code>PUT</code> , <code>PATCH</code>

Query Parameters

Query parameters are key-value pairs added to the end of a URL, separated from the base path by a question mark (`?`). Multiple parameters are strung together using the ampersand (`&`) symbol.

- **Example:** `https://example.com`

- **How they act:** They behave like modifiers or filters. They do not change the fundamental action, but they alter the subset of data returned by the server.
- **Security risk:** Since they appear directly in the URL, **never** use query parameters to send sensitive data like passwords, API keys, or personal identifiers. URLs are routinely saved in browser histories, proxy logs, and server telemetry.

Request Body

The request body is the data payload sent independently of the URL path. It is transmitted in the body stream of the HTTP request.

- **Example:** A JSON object sent to the server:

json

```
{
-   "username": "alice",
-   "email": "alice@example.com",
-   "preferences": { "theme": "dark" }
- }
```

- Use code with caution.
- **How it acts:** It represents the actual resource content you want the server to process, store, or modify.
- **Data Flexibility:** Unlike query strings, a request body easily accommodates heavily nested data schemas, arrays, or uploaded files.

How can we take tuple, list and dictionary as input from the request body in JSON?

- Updating our pydantic code as;

```
from pydantic import BaseModel
from typing import List, Tuple, Dict

class TaskCreate(BaseModel):
    title: str
    description: str
    priority: str
    papa: int
    mama: bool
    nana: float
```

```
nani: List[int]
dadi: Tuple[str, str]
chachi: Dict[str, int]
```

Now since JSON has only these data types;

In JSON

JSON has **no tuple type**. It supports only these data types:

- Object → `{ }`
- Array → `[ ]`
- String → `"text"`
- Number → `123`, `3.14`
- Boolean → `true`, `false`
- Null → `null`

So when we send our tuples in JSON body in circular braces notation `()`, then error occurs;

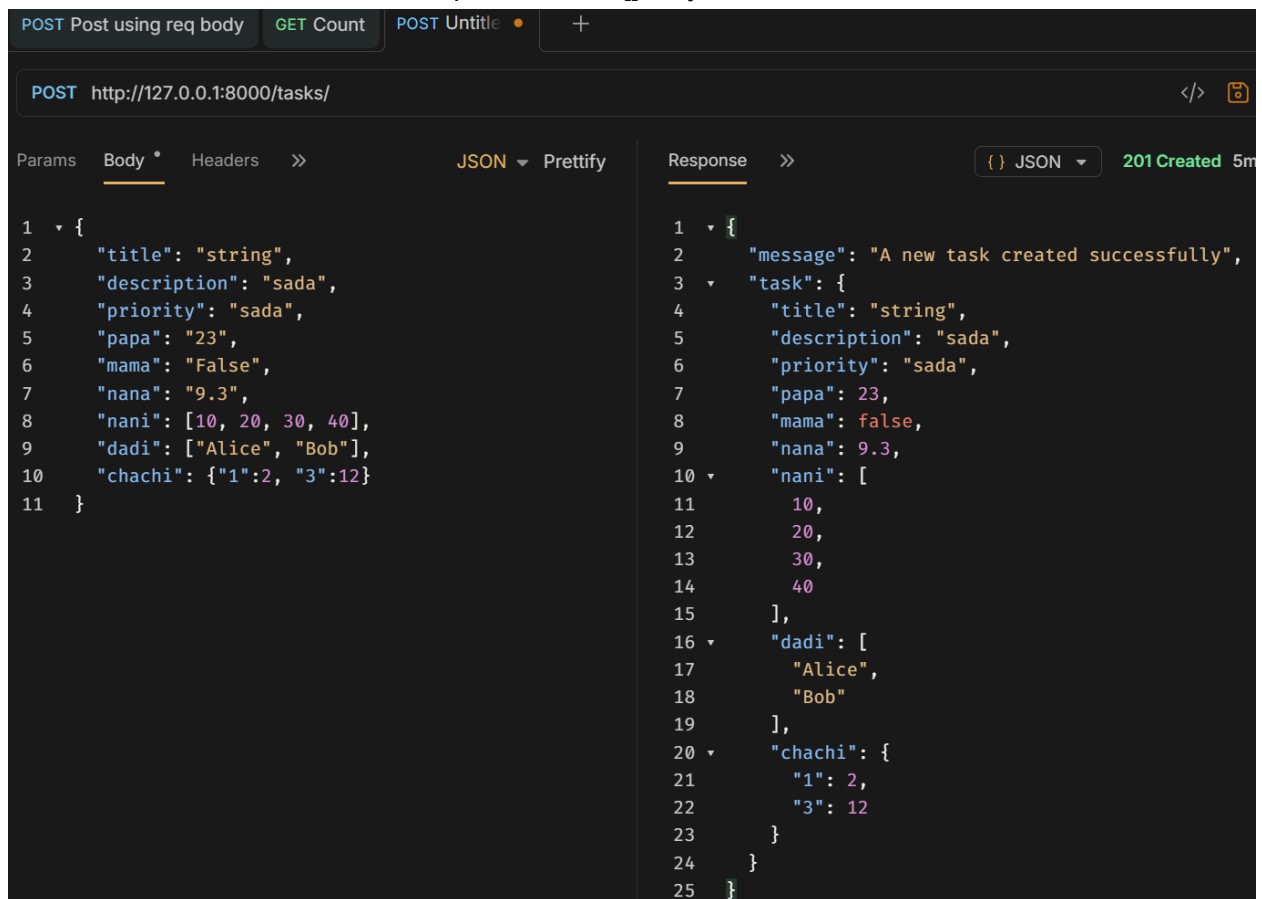
The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8000/tasks/`. The request body is a JSON object with a tuple in the `dadi` field. The response is a 422 Unprocessable Entity error with a detailed JSON message.

```
POST http://127.0.0.1:8000/tasks/

{
  "title": "string",
  "description": "sada",
  "priority": "sada",
  "papa": "23",
  "mama": "False",
  "nana": "9.3",
  "nani": [10, 20, 30, 40],
  "dadi": ("Alice", "Bob"),
  "chachi": {"1": 2, "3": 12}
}
```

```
{
  "detail": [
    {
      "type": "json_invalid",
      "loc": [
        "body",
        160
      ],
      "msg": "JSON decode error",
      "input": {},
      "ctx": {
        "error": "Expecting value"
      }
    }
  ]
}
```

So we need to send them inside square braces [] only;



The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8000/tasks/`. The request body is a JSON object with the following structure:

```
1 {
2   "title": "string",
3   "description": "sada",
4   "priority": "sada",
5   "papa": "23",
6   "mama": "False",
7   "nana": "9.3",
8   "nani": [10, 20, 30, 40],
9   "dadi": ["Alice", "Bob"],
10  "chachi": {"1": 2, "3": 12}
11 }
```

The response is a JSON object with the following structure:

```
1 {
2   "message": "A new task created successfully",
3   "task": {
4     "title": "string",
5     "description": "sada",
6     "priority": "sada",
7     "papa": 23,
8     "mama": false,
9     "nana": 9.3,
10    "nani": [
11      10,
12      20,
13      30,
14      40
15    ],
16    "dadi": [
17      "Alice",
18      "Bob"
19    ],
20    "chachi": {
21      "1": 2,
22      "3": 12
23    }
24  }
25 }
```

But we don't need to worry as when they are received inside the python code, then they won't be perceived as lists, instead they will be perceived as tuples only;



The screenshot shows Python code using Pydantic to validate a dictionary and print the result. The code is as follows:

```
</> Python

from pydantic import BaseModel
from typing import Tuple

class Person(BaseModel):
    dadi: Tuple[str, str]

person = Person.model_validate({
    "dadi": ["Alice", "Bob"]
})

print(person.dadi)
print(type(person.dadi))
```

The output of the code is:

```
</> Python

('Alice', 'Bob')
<class 'tuple'>
```

Why do we use Pydantic instead of manually validating input?

- Pydantic replaces repetitive `if/else` checks by doing the heavy lifting for you. You declare the data's shape using standard Python **type hints**, and Pydantic automatically parses, coerces, and validates incoming data. This approach prevents bugs, improves code readability, and integrates seamlessly with modern frameworks like **FastAPI**.

Here is why you should use Pydantic instead of manual validation:

- **Declarative Schemas:** Instead of writing imperative `if not isinstance(data['age'], int):` blocks, you define a clean model with types like `age: int`.
- **Intelligent Type Coercion:** Pydantic safely converts data types automatically (e.g., transforming a string `"123"` into an integer `123` or `"true"` to a boolean `True`).
- **Detailed Error Reporting:** When validation fails, Pydantic raises a structured `ValidationError` that specifies exactly which fields were wrong and why, gathering all errors at once instead of failing on the first one.
- **Serialization & Generation:** Models can be easily converted to JSON or dictionaries (e.g., `model_dump()`), making data transfer incredibly simple.

What is the purpose of BaseModel?

- The purpose of `BaseModel` in the Pydantic library is to enforce data structure, type safety, and automatic validation in Python. By defining classes that inherit from `BaseModel`, you establish a strict blueprint that incoming or outgoing data must follow, making it ideal for API development.

Why do we keep schemas separate from models?

- Separating schemas from models allows for a clean **separation of concerns** and loose coupling. The **schema** acts as a blueprint defining the structure and validation rules for the data, while the **model** provides the programming interface required to interact with that data.

Why is returning 201 Created better than 200 OK for a successful POST request?

- Returning `201 Created` is better than `200 OK` for a successful `POST` request because **it explicitly communicates at the protocol level that a new resource was successfully generated** rather than just processing data.
- While both indicate success, using the more specific status code enhances API clarity, handles network idempotency gracefully, and aligns with standard RESTful conventions.

What does the task: TaskCreate parameter tell FastAPI to do?

- In FastAPI, the `task: TaskCreate` parameter tells the framework to parse the incoming HTTP request body (usually JSON) as a Pydantic model. It forces FastAPI to validate the input data against the `TaskCreate` schema before executing your endpoint logic.

-----Commit 3 Complete-----

15/07/26

Step 18: Validating data in Request Body using Pydantic